PROPOSED SOLUTION

FloodVision AI

An Explainable Deep Learning Framework for Flood Damage Assessment and Intelligent Rescue Planning

Submitted as part of the ROSP (Research-Oriented Skill Project) requirement

1. Introduction

This document presents the proposed solution for FloodVision AI, developed in response to the limitations identified in the problem statement and literature survey. It describes what is being built, how its parts fit together, and why this combination of capabilities addresses the gap in existing flood-response systems — before the detailed phase-wise implementation plan is executed.

2. Problem Recap

Floods cause significant damage to buildings, roads, and other infrastructure. Rapid, reliable assessment of affected regions is critical for disaster-response authorities to understand the extent of damage and to direct rescue and relief efforts where they are needed most.

Existing tools typically address only one piece of this puzzle — emergency reporting, building-damage mapping, or flood-depth estimation — in isolation. None of the systems reviewed combine flood detection, infrastructure damage assessment, explainable AI, and rescue prioritization within a single, integrated decision-support platform.

3. Proposed Solution Overview

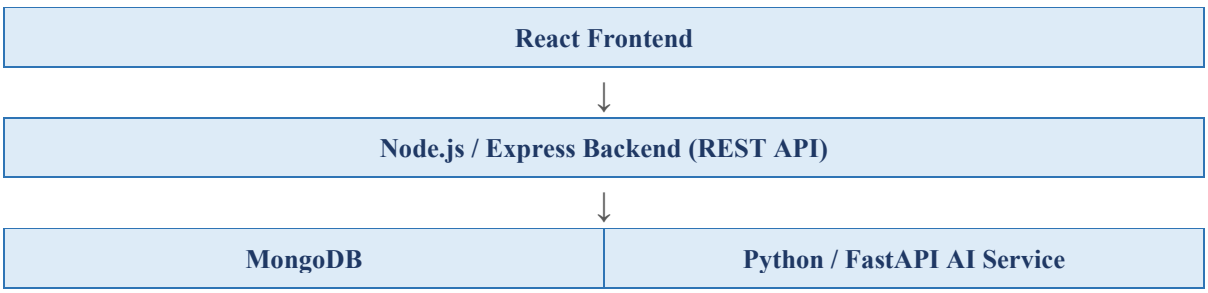
FloodVision AI is proposed as a web-based, AI-powered decision-support platform that unifies six capabilities that are otherwise scattered across separate tools:

- Flood case management for disaster-response teams
- AI-driven flood-region segmentation from satellite/aerial imagery
- AI-driven building damage detection and severity classification
- Explainable AI (XAI) so that every prediction is accompanied by a visual justification
- A transparent, rule-based rescue-prioritization engine
- Interactive GIS visualization and automated report generation

The solution is deliberately built in two independent, loosely-coupled halves — a MERN-based web application and a Python-based AI module — that are integrated only at the end through a dedicated inference service. This keeps development parallelizable and each half independently testable.

4. Solution Architecture

At a high level, the solution follows a four-layer architecture:



↓

PyTorch Deep Learning Models (Flood Segmentation + Damage Assessment)

The frontend never communicates with the AI models directly — all AI requests are brokered through the Node.js backend, which keeps the deep-learning layer swappable and independently versioned.

5. Core Modules of the Proposed Solution

5.1 Flood Case Management Module

Flood Case Management

Lets disaster-response teams organize their work around discrete flood events.

- Create, view, and update flood cases with location, event date, and status
- Role-based access for Administrators, Officers, Analysts, and Viewers
- Central record linking imagery, assessments, and reports to a case

5.2 Image Acquisition & Management Module

Image Upload & Management

Handles the intake of flood-related imagery that feeds the AI pipeline.

- Upload and preview satellite/aerial imagery
- Associate each image with a flood case
- Track image metadata for downstream processing

5.3 AI Analysis Module

Flood Segmentation

Identifies flooded pixels from Sentinel-1 radar imagery.

- Trained on Sen1Floods11 (VV + VH bands, QC ground truth)
- Candidate architectures: U-Net (baseline) or SegFormer
- Output: a pixel-level flood mask with confidence

Building Damage Assessment

Detects buildings and classifies their damage severity from pre/post-disaster imagery.

- Trained on xBD (xView2) pre- and post-event imagery with building annotations
- Uses the dataset's native damage categories (e.g. no damage / minor / major / destroyed)
- Output: per-building damage class and confidence

Explainable AI (XAI)

Attaches a visual justification to every AI prediction, rather than a bare confidence score.

- Grad-CAM or an architecture-appropriate alternative
- Overlay showing which image regions drove the prediction
- Treated as an interpretation aid, not as ground truth

5.4 Rescue Prioritization Engine

Rescue Prioritization Engine

Converts raw AI outputs into an actionable, human-readable priority signal.

- Combines flood severity, building damage, and road/accessibility signals
- Rule-based and transparent, so its reasoning can be audited and explained
- Produces a HIGH / MEDIUM / LOW priority score per area

5.5 GIS Visualization Module

Interactive Map

Presents all assessment results geographically, using React Leaflet + OpenStreetMap.

- Flooded areas and damaged structures plotted on an interactive map
- Priority zones highlighted for rapid field decision-making
- Map view linked back to the underlying case and assessment records

5.6 Reporting Module

Assessment Reports

Packages the full assessment into a shareable report.

- Case information, location, and flood statistics
- Building and infrastructure damage summary
- Severity, rescue priority, AI confidence, and supporting visualizations

6. End-to-End Data Flow

The solution's working principle, from image upload to actionable output, follows this path:

User Uploads Flood Imagery

↓

Backend Creates Assessment Record

↓

AI Service Runs Flood Segmentation + Damage Assessment

↓

XAI Generates Visual Explanations

↓

Priority Engine Computes Rescue Priority

↓

Structured Results Stored in MongoDB

↓

Dashboard, Map & Report Rendered to User

7. Technology Stack

Layer	Technology
Frontend	React.js, TypeScript, TailwindCSS
Backend	Node.js, Express.js, TypeScript
Database	MongoDB
AI Development Environment	Python, Google Colab (GPU)
Deep Learning	PyTorch, OpenCV, U-Net / SegFormer / YOLO-based models
Explainable AI	Grad-CAM and other architecture-appropriate XAI techniques
AI Serving	Python, FastAPI
Mapping / GIS	React Leaflet + OpenStreetMap
API Communication	REST APIs

8. Why This Solution — Novelty and Advantages

The proposed system's advantage is not a claim of higher raw model accuracy than any single existing tool. Its distinguishing strength is integrated functionality — combining detection, assessment, explanation, prioritization, and visualization in one workflow, rather than requiring responders to stitch together multiple disconnected tools.

Feature	AI Disaster Response	Building Damage Mapping	FloodVision VLM	FloodVision AI
Disaster reporting	✓	—	—	✓
GIS visualization	✓	✓	—	✓
Satellite imagery	—	✓	—	✓
Flood detection	Limited	—	✓	✓
Flood segmentation	—	—	—	✓
Building damage assessment	—	✓	—	✓
Road/infrastructure assessment	—	—	—	✓
Explainable AI	—	—	—	✓
Rescue prioritization	✓	—	—	✓
Integrated decision support	Partial	Partial	Partial	✓

9. Development Approach Summary

The solution will be built in two independently-developed halves, integrated at the end:

Web Application (MERN)

The complete case-management, upload, dashboard, and GIS experience is built and tested first, using mock AI results, so the web platform can be validated independently of model readiness.

AI Module (Python / Colab)

In parallel, flood segmentation and building-damage models are developed, trained, and evaluated independently in Google Colab against Sen1Floods11 and xBD, complete with XAI and evaluation metrics.

Integration

The two halves are joined through a Python/FastAPI AI service that the Node.js backend calls, replacing the mock assessment logic with real model inference — full detail of this sequencing is covered in the separate Implementation Plan.

10. Expected Benefits

1. Faster, more consistent flood-damage assessment than manual review.
2. A single platform instead of multiple disconnected tools for reporting, mapping, and assessment.
3. Greater trust in AI outputs through visual explanations rather than opaque confidence scores.
4. Transparent, auditable rescue prioritization that responders can question and understand.
5. Geographic context for every decision, via the integrated GIS map.

6. Ready-to-share assessment reports for coordination between agencies.

11. Conclusion

FloodVision AI proposes to close the integration gap left by existing single-purpose tools by combining flood segmentation, building damage assessment, explainable AI, rescue prioritization, and GIS visualization into one decision-support platform. The following Implementation Plan translates this proposed solution into a concrete, phase-wise build sequence.